04 — Planning the Training of a Very Large Model

How to budget memory and compute for a model with a hundred billion parameters, how to split its training across many GPUs, and what goes wrong at that scale.

Reading time: about 55 minutes. You need: modules 00 to 03, and a rough idea of GPU memory and network links. You do not need: access to a cluster. Everything in the exercise runs on one laptop.

Twenty GPUs before the first token

Take a language model with 100 billion parameters. Training it with the Adam optimiser costs 16 bytes of memory for every parameter, as section 1 explains. That comes to 1.6 terabytes.

An NVIDIA H100 GPU has 80 GB of memory. So merely holding the training state takes 20 of them. That is before storing any intermediate results, and before processing a single token of data.

Compute tells a similar story. GPT-3, with 175 billion parameters trained on 300 billion tokens, needed about 3.15×10^{23} floating-point operations. Section 2 derives that figure. Even on 1,024 H100s running at a realistic 40% of their peak speed, it takes about nine days.

So the question "how would you train a 100B model?" is not really about a software library. It starts with arithmetic. How much memory is needed, and how much compute? And how can the work be divided so that it fits, without the GPUs spending all their time talking to each other? This article builds that arithmetic, one piece at a time.

What you will learn

By the end of this article, you will be able to:

- estimate the memory needed for weights, gradients, optimiser state and activations;
- estimate training compute and time with the formula $C \approx 6ND$;
- explain data, tensor and pipeline parallelism, and ZeRO, including what each one costs;
- combine them for a real cluster, taking network speeds into account;
- explain how to choose a model's size for a compute budget, and what breaks at scale.

Sections 1 and 2 build the budgets. Sections 3 to 5 cover the three main ways to split training. Section 6 combines them, and section 7 covers failures and model size. A recap and the exercise briefing follow at the end. The models and letters used throughout are tabulated after the recap, for looking up rather than reading.

---

1. Memory: sixteen bytes per parameter

Training with Adam in **mixed precision**, meaning a mix of 16-bit and 32-bit numbers, stores five things for every parameter:

What is stored	Format	Bytes per parameter	What it is for
weights	16-bit	2	read by the forward pass
gradients	16-bit	2	produced by the backward pass
master weights	32-bit	4	a precise copy that receives updates

What is stored	Format	Bytes per parameter	What it is for
Adam's first moment	32-bit	4	a running average of gradients
Adam's second moment	32-bit	4	a running average of squared gradients
total		**16**	

Sixteen bytes per parameter is the number to remember. For 100 billion parameters it gives 1.6 TB, and it is the first thing to write on a whiteboard.

Why keep a 32-bit copy?

The master weights look wasteful, but they solve a real problem. A 16-bit number cannot represent small changes. Here is what happens when you add small updates to a weight of 0.02:

Format	Does $0.02 + 10^{-4}$ change?	$0.02 + 10^{-5}$?	$0.02 + 10^{-6}$?
float16	yes	yes	no
bfloat16	yes	no	no
float32	yes	yes	yes

A learning rate of 10^{-4} times a small gradient easily produces an update of 10^{-6}. In bfloat16, that update rounds away to nothing, and the weight never moves. So the optimiser applies updates to a 32-bit copy, and a 16-bit copy is made from it for the fast forward and backward passes.

A useful consequence follows. Pure 32-bit training also costs 16 bytes per parameter: 4 for weights, 4 for gradients and 8 for Adam. Mixed precision saves time and activation memory, not this static state. For comparison, gradient descent with momentum needs 12 bytes, and plain gradient descent 8.

---

2. Activations and compute

Activation memory grows with the square of sequence length

Activations are the intermediate values from the forward pass that the backward pass needs. Their size depends on the batch and the sequence length, not only on the model. Korthikanti et al. (2022) give this estimate, in bytes, for a transformer stored in 16 bits:

$$s \cdot b \cdot h \cdot L \cdot \left(34 + 5\,\frac{a\,s}{h}\right)$$

Here s is the sequence length, b the micro-batch size, h the hidden size, L the number of layers and a the number of attention heads.

Read it in two parts. The 34 covers everything that grows in proportion to the sequence length. The second term, $5as/h$, contains another factor of s, so overall it grows with s^2. It comes from the attention score matrix, which compares every token with every other token.

Neither number is arbitrary. Both are counts of saved tensors, and the paper lists them. Each number below is bytes per token, per sequence in the micro-batch, per layer. A 16-bit value costs 2 bytes and a dropout mask costs 1.

Block	What is saved for the backward pass	Bytes
attention	input shared by the Q, K and V projections	2
	Q and K, kept for the score matmul	4
	V, kept for the value matmul	2
	input of the output projection	2
	dropout mask	1
MLP	input of the first linear layer	2
	input of the second linear layer, which is $4h$ wide	8
	input of the GELU, also $4h$ wide	8
	dropout mask	1
LayerNorm	the input of each of the two LayerNorms	4
total		**34**

The three tensors whose size involves s^2 sit in the middle of attention: the softmax output ($2as^2b$), its dropout mask (as^2b) and the dropout output ($2as^2b$). Those five units are the 5 in $5as/h$. Every one of them is $s \times s$ for each head, which is why they, and nothing else, grow with the square.

With GPT-3's shape (96 layers, hidden size 12,288, 96 heads) and one sequence per micro-batch:

Sequence length	Activation memory	With selective recomputation	Share from attention scores
2,048	256 GiB	76 GiB	70%
4,096	873 GiB	153 GiB	83%
8,192	3,186 GiB	306 GiB	90%
16,384	12,132 GiB	612 GiB	95%

The last two columns come straight from the two parts of the formula. Selective recomputation keeps the 34 term and recomputes the rest, so its column is $34\,sbhL$ bytes. The share from attention is the second term divided by the whole, $5as/h$ over $34 + 5as/h$.

Here is the first row in full, so the rest can be checked the same way. At $s = 2{,}048$:

$$5\frac{as}{h} = \frac{5 \times 96 \times 2{,}048}{12{,}288} = 80 \qquad \text{so} \qquad 34 + 80 = 114$$

$$2{,}048 \times 1 \times 12{,}288 \times 96 \times 114 = 2.75 \times 10^{11} \text{ bytes} = 256 \text{ GiB}$$

The attention share is 80 over 114, which is 70%. Selective recomputation keeps only the 34, giving $256 \times 34/114 = 76$ GiB.

Quadrupling the sequence from 2,048 to 8,192 tokens multiplies activation memory by 12.4. At long sequences, the attention scores dominate everything. That is the economic reason FlashAttention exists (module 07): it never stores that matrix.

Recomputation trades time for memory

Activation recomputation, also called gradient checkpointing, stores only the input to each layer. During the backward pass it recomputes everything else. In theory this costs about one extra forward pass, roughly a third more compute, in exchange for a large memory saving.

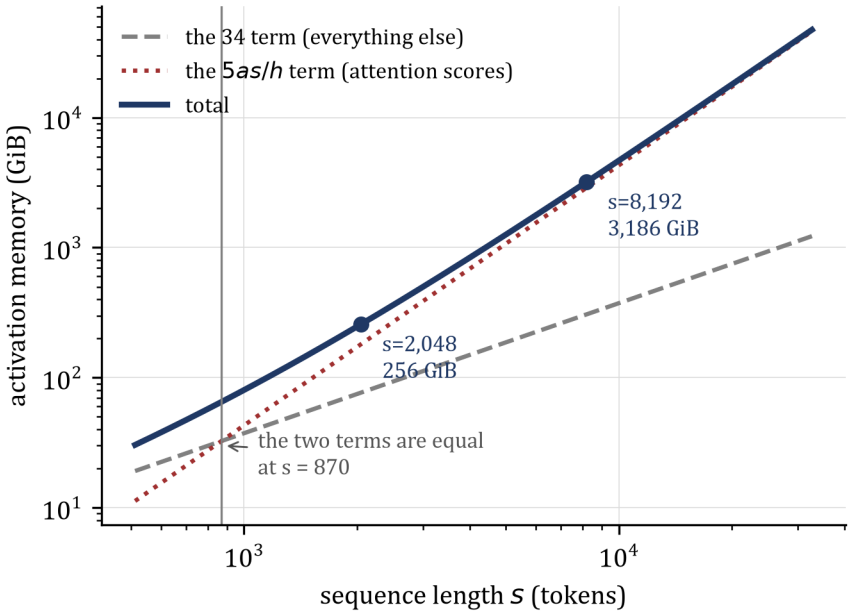


Figure 1: The two terms of the activation formula, for GPT-3's shape with one sequence per micro-batch. Below about 900 tokens the ordinary activations dominate. Above it the attention scores do, and they keep doubling twice as fast as the sequence length.

Selective recomputation is the modern default. It recomputes only the attention scores, which are cheap to recompute and expensive to store. The third column of the table shows its effect.

An honest note on measurement: in the exercise's small CPU test, checkpointing cut stored activations by more than half, but made each training step about 2.6 times slower. The layers there are tiny, so Python overhead dominates. On large models running on GPUs, the cost is much closer to one extra forward pass.

Compute: C ≈ 6ND

The standard estimate for training compute is easy to derive. The forward pass does about one multiplication and one addition per weight, so it costs about $2N$ FLOPs per token for N parameters.

The backward pass costs about twice that, and the reason is worth knowing. Each layer computes $y = Wx$ in the forward direction, which is one matrix multiplication. Going backwards it must produce two things. The first is the gradient with respect to its input, $\bar{x} = W^\top \bar{y}$, which the layer before it needs in order to continue. The second is the gradient with respect to its own weights, $\bar{W} = \bar{y}\, x^\top$, which the optimiser needs. Each is a matrix multiplication of the same size as the forward one.

One matmul forward, two backward: the backward pass costs twice the forward. So each token costs about $6N$ FLOPs, and training on D tokens costs:

$$C \approx 6ND$$

With full recomputation, add one more forward pass: $C \approx 8ND$.

For GPT-3, $6 \times 175 \times 10^9 \times 300 \times 10^9 = 3.15 \times 10^{23}$ FLOPs. An H100 delivers at most about 990×10^{12} 16-bit FLOPs per second, counting dense arithmetic. Marketing figures that count sparsity are twice as large, and no training run achieves them.

The time follows by division. The cluster supplies $1{,}024 \times 990 \times 10^{12} \times 0.4 = 4.1 \times 10^{17}$ FLOPs per second, so the run takes $3.15 \times 10^{23} / 4.1 \times 10^{17} = 7.8 \times 10^{5}$ seconds, which is 9.0 days.

That 40% is called **model FLOPs utilisation**, or MFU: the FLOPs actually achieved, divided by the hardware's peak. Well-tuned large training runs reach 40–55%.

---

3. How GPUs share data: data parallelism and ZeRO

This section answers two questions at once: how data physically reaches the GPU that needs it, and what each level of ZeRO actually changes.

Copy the model, split the batch

The simplest way to use many GPUs is **data parallelism**. Every GPU holds a full copy of the model and processes a different slice of the batch. After the backward pass, the GPUs average their gradients, so every copy takes the same update.

The averaging step is called an **all-reduce**: every GPU ends up holding the combined result. The next three subsections open it up, because its cost sets the limit on this whole approach.

A common mistake is to *sum* the gradients instead of averaging them. Each GPU's loss is already an average over its own slice, so summing multiplies the effective learning rate by the number of GPUs. The exercise shows the result is wrong by exactly that factor.

Three kinds of data cross between GPUs

Every scheme in this article moves something between GPUs, but not the same something. Keeping the three apart makes the rest of the article easier to hold in your head.

What moves	When	Which scheme needs it	How big it is
gradients	once per optimiser step	data parallelism, every ZeRO stage	one copy of the model, 2 bytes per parameter
weights	before each layer runs	ZeRO-3 and FSDP	one copy of the model, twice per step
activations	at every split layer	tensor parallelism	$s \times b \times h \times 2$ bytes per transfer
activations	at each stage boundary	pipeline parallelism	the same, once per micro-batch
tokens	at every expert layer	expert parallelism	the tokens routed elsewhere

The first two grow with the *model*. The last three grow with the *batch and the sequence length*. That single difference explains why the schemes fail in different ways as you scale one or the other.

The five collective operations

GPUs exchange data through **collectives**: operations that every GPU in a group runs together, rather than one GPU reading another's memory. Five of them cover everything here. In the table, n is the number of GPUs in the group and "size" is the size of the whole tensor.

Collective	Each GPU starts with	Each GPU ends with	Bytes it sends
broadcast	one GPU holds x	a copy of x	about the size
all-reduce	its own x_i	the sum $\sum_i x_i$	$2(n-1)/n$ of it
reduce-scatter	its own x_i	its own slice of that sum	$(n-1)/n$ of it
all-gather	one slice	every slice	$(n-1)/n$ of it
all-to-all	n slices, one per GPU	the slices addressed to it	$(n-1)/n$ of it

One relationship between them matters more than the rest:

$$\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}$$

That is how an all-reduce is implemented, and it is why it costs twice what either half costs. It is also, as the ZeRO subsections show, the reason some of ZeRO's savings are free.

Inside an all-reduce: the ring

Averaging a gradient across n GPUs sounds like it should need a central collector. The ring algorithm needs none, and sends a near-constant amount per GPU however large n grows.

Here is the whole thing for four GPUs, one step at a time.

The set-up. Every GPU has just computed a gradient for the *whole* model, from its own slice of the batch. Four GPUs therefore hold four complete gradients, which differ because the data differed. The job is to add them up.

Cut each of those gradients into four equal pieces by parameter number, and call the pieces chunks. Chunk 0 is the first quarter of the parameters, chunk 1 the second, and so on:

```
GPU 0's gradient:  [ chunk 0 | chunk 1 | chunk 2 | chunk 3 ]  computed from GPU 0's data
GPU 1's gradient:  [ chunk 0 | chunk 1 | chunk 2 | chunk 3 ]  computed from GPU 1's data
GPU 2's gradient:  [ chunk 0 | chunk 1 | chunk 2 | chunk 3 ]  computed from GPU 2's data
GPU 3's gradient:  [ chunk 0 | chunk 1 | chunk 2 | chunk 3 ]  computed from GPU 3's data
                       |
                   `-- four versions of this quarter exist; they must be added together
```

Sixteen pieces, then, and four separate addition jobs. Seat the GPUs in a ring, so each one sends only to its right-hand neighbour.

How to read the grids below. The grid is a picture of those sixteen pieces:

- **A row is one GPU's memory.** Everything on GPU 2's row is stored on GPU 2. Every GPU keeps all four cells of its row throughout, so nothing in the grid is ever empty.
- **A column is one quarter of the model**, held as four separate copies, one per GPU.
- **A cell says how complete that particular copy is**, by listing the contributions already added into it.

So `2` means a copy holding GPU 2's own contribution and nothing else. `023` means a copy into which the contributions of GPUs 0, 2 and 3 have been summed. A copy is finished when all four have been added, which the grid writes as `[0123]`.

At the start, no GPU has received anything, so every copy holds exactly one contribution: its own.

```
start: every GPU has its own piece of all four chunks
          chunk 0  chunk 1  chunk 2  chunk 3
  GPU 0 |  0        0        0        0
  GPU 1 |  1        1        1        1
  GPU 2 |  2        2        2        2
  GPU 3 |  3        3        3        3
```

Read one cell to be sure of the convention. Row `GPU 1`, column `chunk 2`, holds `1`: that is GPU 1's own version of the model's third quarter, with nothing else added yet. The grid after step 1 shows the same cell still reading `1`, while `chunk 0` on that row has become `01`.

Phase 1: reduce-scatter, three steps

In each step, every GPU sends **one** chunk to its right-hand neighbour, which adds the arriving copy into its own. Every GPU sends in every step, and the chunk it sends shifts back by one place each time. At step t, GPU i sends chunk $i - t$, counting round.

Step 1. GPU 0 sends chunk 0, GPU 1 sends chunk 1, GPU 2 sends chunk 2, GPU 3 sends chunk 3:

```
after step 1
          chunk 0  chunk 1  chunk 2  chunk 3
  GPU 0 |  0        0        0        03
  GPU 1 |  01       1        1        1
  GPU 2 |  2        12       2        2
  GPU 3 |  3        3        23       3
```

Each GPU now holds exactly one two-way sum: the chunk its left-hand neighbour sent, added into its own copy. GPU 1 holds `01` in the chunk 0 column, because chunk 0 is what GPU 0 sent. Nothing has been duplicated, and each of the four partial sums sits in exactly one place.

Step 2. Each GPU passes on the sum it just received, so those four sums move one place further round the ring. GPU 0 sends chunk 3, GPU 1 sends chunk 0, GPU 2 sends chunk 1, GPU 3 sends chunk 2:

```
after step 2
          chunk 0  chunk 1  chunk 2  chunk 3
  GPU 0 |  0        0        023      03
  GPU 1 |  01       1        1        013
  GPU 2 |  012      12       2        2
  GPU 3 |  3        123      23       3
```

Step 3. The same again, and each three-way sum collects the one contribution it still lacks:

```
after step 3
          chunk 0  chunk 1  chunk 2  chunk 3
  GPU 0 |  0       [0123]    023      03
  GPU 1 |  01       1       [0123]    013
  GPU 2 |  012      12       2       [0123]
  GPU 3 | [0123]    123      23       3
```

Three steps, and four finished chunks exist: GPU 0 owns the finished chunk 1, GPU 1 owns chunk 2, GPU 2 owns chunk 3 and GPU 3 owns chunk 0. Every other cell is a partial sum that is now useless, and is dropped.

Notice what has *not* happened. No GPU holds the complete gradient. Each holds one quarter of it, correctly summed. That is precisely a reduce-scatter.

It is also where ZeRO-1 and ZeRO-2 stop. A GPU that updates only its own slice of the weights needs only its own slice of the gradient, so the second phase below is wasted work for them.

Phase 2: all-gather, three more steps

The second phase spreads those four finished chunks to everyone. It is the same ring and the same rhythm, except that arriving chunks are stored rather than added, because they are already complete.

After the first step, every GPU holds two finished chunks:

```
after step 1 of the all-gather
          chunk 0  chunk 1  chunk 2  chunk 3
  GPU 0 | [0123]   [0123]    023      03
  GPU 1 |  01      [0123]   [0123]    013
  GPU 2 |  012      12      [0123]   [0123]
  GPU 3 | [0123]    123      23      [0123]
```

Two more steps take each GPU from two finished chunks to three, and then to four:

```
after step 3 of the all-gather
          chunk 0  chunk 1  chunk 2  chunk 3
  GPU 0 | [0123]   [0123]   [0123]   [0123]
  GPU 1 | [0123]   [0123]   [0123]   [0123]
  GPU 2 | [0123]   [0123]   [0123]   [0123]
  GPU 3 | [0123]   [0123]   [0123]   [0123]
```

Every GPU now holds the summed gradient in full, which is what an all-reduce promises.

Count the traffic. Each GPU sent $1/n$ of the gradient in each of $2(n-1)$ steps:

$$\frac{2(n-1)}{n} \times \text{the gradient}$$

With 4 GPUs that is 1.5 copies, with 64 it is 1.97, and with 1,024 it is 1.998. The cost per GPU rises towards two copies and stops there, however many GPUs join. That is the reason data parallelism scales to thousands of GPUs, while the schemes in sections 4 and 5 do not.

Communication hides behind the backward pass

An all-reduce need not wait for the backward pass to end. Gradients become final layer by layer, from the output backwards. So PyTorch's `DistributedDataParallel` collects them into **buckets** of about 25 MB, and starts each bucket's all-reduce as soon as it fills. Most of the traffic is already gone by the time the first layer's gradient arrives.

So the question is never "how many bytes" alone. It is how those bytes compare with the compute they can hide behind. Take the 100-billion-parameter model on 64 GPUs. One copy of it in 16-bit format is 200 GB, so an all-reduce moves about 394 GB per GPU. Over an inter-node link carrying 50 GB per second, that is 7.9 seconds of traffic per step:

Tokens per GPU per step	Compute	Traffic	Traffic as a share of compute
1,024	1.6 s	7.9 s	508%
8,192	12.4 s	7.9 s	63%
65,536	99.3 s	7.9 s	8%

The compute column is $6N$ FLOPs per token, at 40% of an H100's peak. Small steps drown in communication; large ones hide it entirely. This is why large runs use batches of millions of tokens, and it is the second reason for gradient accumulation below.

ZeRO: stop storing everything everywhere

Data parallelism's weakness is memory, not speed: every GPU stores all 16 bytes per parameter. **ZeRO**, the zero redundancy optimiser, fixes this by splitting the stored state across the GPUs, in three stages. For a 100B model on 64 GPUs:

Stage	What is split across GPUs	Bytes per parameter	Memory per GPU	Extra communication
none	nothing	16	1,600 GB	none
ZeRO-1	optimiser state	4.19	419 GB	none
ZeRO-2	plus gradients	2.22	222 GB	none
ZeRO-3	plus the weights	0.25	25 GB	gather each layer's weights before use

Each row is the same sum with more terms divided by the 64 GPUs. A full copy is 2 bytes of weights, 2 of gradients and 12 of optimiser state. ZeRO-1 keeps $2 + 2 + 12/64 = 4.19$, ZeRO-2 keeps $2 + (2 + 12)/64 = 2.22$, and ZeRO-3 splits everything, leaving $16/64 = 0.25$. Multiply by 100 billion parameters for the memory column.

ZeRO-3 is what PyTorch's **FSDP** (fully sharded data parallel) implements. It brings memory per GPU down in proportion to the number of GPUs.

What each stage does during one step

The stages differ in which collective runs, and when. Write ψ for one copy of the model in 16-bit format, which is 200 GB for 100 billion parameters.

Stage	Forward pass	Backward pass	Optimiser step	Moved
plain	every GPU holds every weight	every GPU forms every gradient	all-reduce the gradients, then every GPU repeats the same update on the whole model	2ψ

Stage	Forward pass	Backward pass	Optimiser step	Moved
ZeRO-1	unchanged	unchanged	reduce-scatter the gradients; each GPU updates only its own slice of the master weights and Adam moments; all-gather the updated 16-bit weights	2ψ
ZeRO-2	unchanged	each layer's gradient is reduced as soon as it is final, and the GPUs that do not own it drop it	as ZeRO-1	2ψ
ZeRO-3	before each layer runs, all-gather that layer's weights, use them, free them again	gather the layer's weights a second time, compute, then reduce-scatter its gradients	each GPU updates its own slice, and nothing is gathered at the end, because the next forward pass gathers afresh	3ψ

Now the trade-off is visible. **ZeRO-1 and ZeRO-2 move exactly as much as plain data parallelism.** An all-reduce was always a reduce-scatter followed by an all-gather, and these two stages do no more than slip the optimiser step between the halves. The memory they save is free.

Only ZeRO-3 pays, and it pays half as much again. Each layer's weights are gathered twice per step, once going forward and once coming back, on top of the gradient traffic. For the model on 64 GPUs that is 591 GB per GPU per step instead of 394 GB. At 50 GB per second, it is 11.8 seconds instead of 7.9.

Stage	Memory per GPU	Traffic per GPU per step	At 50 GB/s
none	1,600 GB	394 GB	7.9 s
ZeRO-1	419 GB	394 GB	7.9 s
ZeRO-2	222 GB	394 GB	7.9 s
ZeRO-3	25 GB	591 GB	11.8 s

Four practical consequences follow:

- **Use the lowest stage that fits.** ZeRO-1 and ZeRO-2 are nearly free, so the real decision is whether you need stage 3 at all.
- **Stage 3 must prefetch.** Its gathers sit on the critical path, so an implementation that waits for layer i's weights before starting layer i stalls. FSDP fetches layer $i + 1$ while layer i computes, which hides most of the delay.
- **Small layers hurt.** Each gather is a separate collective, and a collective has a fixed cost of a few microseconds. Many small layers turn into many small messages, where latency rather than bandwidth decides the time.
- **Shard inside a server, replicate across servers.** The usual compromise, called hybrid sharding, keeps stage-3 traffic on NVLink and leaves only the ordinary gradient all-reduce crossing the slower links.

Gradient accumulation

A cheaper cousin is **gradient accumulation**. Run k small batches before each update, adding up their gradients. The result equals one batch that is k times larger, with no extra memory.

Two details are easy to get wrong. Divide each small batch's loss by k, or the effective learning rate grows k times. And with data parallelism, skip the all-reduce for all but the last small batch; PyTorch's `no_sync()` does this.

Skipping those all-reduces is the point, not an optimisation detail. It multiplies the tokens processed per communication by k, which moves a run down the traffic table above. Accumulation buys memory and hides communication at the same time.

---

4. Tensor parallelism: splitting the matrices

When a single layer is too big for one GPU, split the matrices themselves. This is **tensor parallelism**. The standard scheme comes from NVIDIA's Megatron-LM, and it is easiest to see on the MLP, $Y = \mathrm{GELU}(XA)B$, with two GPUs.

1. Split A by columns, into A_1 and A_2. Each GPU computes $\mathrm{GELU}(XA_i)$ on its own half. GELU works on each number separately, so no communication is needed yet.
2. Split B by rows, into B_1 and B_2. Each GPU computes $\mathrm{GELU}(XA_i)B_i$, which is part of the final sum. One all-reduce adds the two parts together.

Why columns first? If A were split by rows, each GPU would hold part of a sum that must be completed *before* the GELU, because $\mathrm{GELU}(u + v) \neq \mathrm{GELU}(u) + \mathrm{GELU}(v)$. That would need an extra all-reduce.

The shapes make the scheme concrete. With hidden size h, a sequence of s tokens and two GPUs:

Quantity	Unsplit shape	On each GPU
A	$h \times 4h$	$h \times 2h$, split by columns
$\mathrm{GELU}(XA_i)$	$s \times 4h$	$s \times 2h$, no communication yet
B	$4h \times h$	$2h \times h$, split by rows
$\mathrm{GELU}(XA_i)B_i$	$s \times h$	$s \times h$, but only part of the sum

Note what crosses the link: the last row, an activation tensor. Its size depends on the batch and the sequence length, not on how many parameters the model has.

The result is one all-reduce in the forward pass and one in the backward pass, per MLP. Attention splits the same way, by heads. Heads are already independent, so each GPU can own a subset of them, and the output projection is split by rows.

Why tensor parallelism stays inside one machine

These all-reduces carry whole activation tensors, and there are four per layer: two forward and two backward. With one sequence of 8,192 tokens and hidden size 12,288, each one moves $8{,}192 \times 12{,}288 \times 2$ bytes, which is 201 MB.

Within one server, GPUs connect through NVLink at about 900 GB per second, so each all-reduce takes about 201/900,000 seconds, or 0.22 ms. Between servers, InfiniBand manages about 50 GB per second, so each takes about 4 ms. For a 96-layer model, that is 384 all-reduces per step: about 86 ms over NVLink,

but 1.5 seconds over InfiniBand. So tensor parallelism is kept inside one server, usually across at most 8 GPUs.

---

5. Pipeline parallelism: splitting the layers

Pipeline parallelism splits the model by layers. GPU 0 holds the first layers, GPU 1 the next, and so on. Only the activations at each boundary travel between GPUs, so slower links between servers are fine.

The cost is idle time. A GPU cannot start its part of the backward pass until the GPUs after it have finished. To reduce the waiting, the batch is split into **micro-batches** that flow through the pipeline one after another. The idle time that remains is called the **bubble**.

The exercise simulates the schedule directly. Here it is for 3 stages and 4 micro-batches. `F2` means the forward pass of micro-batch 2, `B2` its backward pass, and `..` means the GPU is idle:

```
GPipe schedule
  stage 0 | F0 F1 F2 F3 .. .. .. .. B0 B1 B2 B3
  stage 1 | .. F0 F1 F2 F3 .. .. B0 B1 B2 B3 ..
  stage 2 | .. .. F0 F1 F2 F3 B0 B1 B2 B3 .. ..
```

Count the idle slots: 12 out of 36, or one third. The formula follows from the chart rather than the other way round. Each stage performs $2m$ units of work, m forward and m backward. The whole schedule lasts $2(m + p - 1)$ slots, because the first micro-batch takes $p - 1$ slots to reach the last stage, and the same delay is paid again on the way back. So each stage idles for $2(p - 1)$ slots, and the fraction is:

$$\frac{2(p-1)}{2(m+p-1)} = \frac{p-1}{m+p-1}$$

For the chart above, $p = 3$ and $m = 4$: each row is $2(4 + 2) = 12$ slots long, 8 of them busy and 4 idle, and the fraction is $2/6$, one third.

A better schedule: 1F1B

The **1F1B** schedule (one forward, one backward) starts backward passes as early as possible:

```
1F1B schedule
  stage 0 | F0 F1 F2 .. .. B0 F3 B1 .. B2 .. B3
  stage 1 | .. F0 F1 .. B0 F2 B1 F3 B2 .. B3 ..
  stage 2 | .. .. F0 B0 F1 B1 F2 B2 F3 B3 .. ..
```

It has exactly the same number of idle slots. Its advantage is memory. Each forward pass stores activations until the matching backward pass runs, and 1F1B keeps that gap short. For 8 stages:

Micro-batches m	Bubble	GPipe: most activations stored	1F1B: most activations stored
8	0.467	8	8
16	0.304	16	8
32	0.179	32	8
64	0.099	64	8

The last two columns count micro-batches whose activations are still waiting for their backward pass. GPipe runs every forward pass first, so that count is m. 1F1B starts backward passes as soon as it can, so the count never exceeds the number of stages, which is 8 here.

So more micro-batches shrink the bubble, and 1F1B keeps memory flat while you add them. A common rule of thumb is to use at least four micro-batches per stage. A further variant, **interleaved 1F1B**, gives each GPU several non-adjacent groups of layers. It cuts the bubble further, at the cost of more communication.

---

6. Other splits, and putting them together

Two further techniques complete the picture:

- **Sequence parallelism** splits along the sequence for the operations that treat tokens independently, such as LayerNorm and dropout. It saves activation memory that tensor parallelism leaves duplicated. **Ring Attention** extends the idea to attention itself, passing keys and values around a ring of GPUs.
- **Expert parallelism** applies to mixture-of-experts models, where each token uses only a few of many MLP "experts". Different experts live on different GPUs. The engineering challenges are the all-to-all communication and keeping the load balanced across experts.

Before combining them, it is worth seeing the three main schemes side by side. Everything in this table has been derived in sections 3 to 5.

	data parallel	tensor parallel	pipeline parallel
What is split	the batch	every weight matrix	the layers
What is replicated	all weights, unless ZeRO	nothing inside a layer	nothing
What crosses GPUs	gradients	activations	activations at the boundaries
How often	once per step	four all-reduces per layer per step	once per micro-batch per boundary
Traffic grows with	the model size	batch × sequence × width	batch × sequence × width
Link it needs	tolerable over InfiniBand	NVLink only	tolerable over InfiniBand
Effect on memory	none, or divided by n with ZeRO	weights and activations divided by n	weights divided by p
Typical degree	tens to thousands	2 to 8	4 to 16
What it costs you	traffic that grows with the model	the fastest link in the machine	the bubble

A typical recipe for a model above 100 billion parameters combines three axes, often called **3D parallelism**:

```
tensor parallel   = 8       within one server, over NVLink
pipeline parallel = 8 to 16 across servers, small transfers
data parallel     = the rest, with ZeRO-1 for optimiser state
```

The three degrees multiply to give the total number of GPUs. Their placement matters too: the GPUs that exchange the most data, the tensor-parallel group, must be the ones with the fastest link.

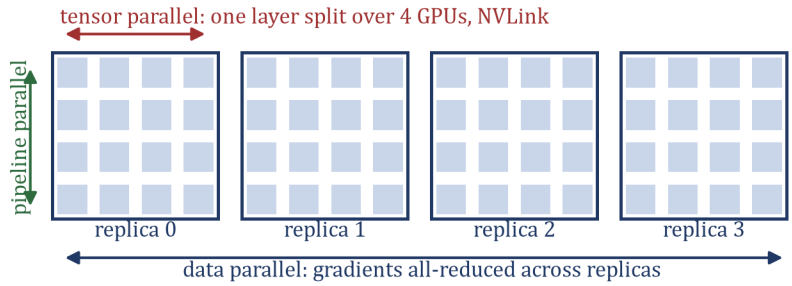


Figure 2: Sixty-four GPUs addressed by three axes at once, here 4 tensor-parallel by 4 pipeline stages by 4 data-parallel replicas. Each square is one GPU. A row inside a replica shares one layer over NVLink; a column holds different layers; the replicas exchange only gradients.

> **In the interview:** start with the 16-bytes-per-parameter budget, then justify each parallelism choice by a network speed. "Tensor parallel inside a server because NVLink is fast; pipeline across servers because only boundary activations travel" is the reasoning that is being tested.

---

7. What breaks at scale, and how big to make the model

Things that go wrong

An answer that stops at parallelism is incomplete. Follow-up questions usually cover failures:

- **Hardware fails.** With a thousand GPUs running for a month, failures come every few hours, not every few weeks. So save checkpoints often, restart automatically, and order the data deterministically so a restart sees the same tokens.
- **The loss spikes.** Sudden jumps in loss are normal at scale. Common fixes are to skip the bad batch, lower the learning rate, roll back to an earlier checkpoint, and prefer bfloat16 to float16.
- **One GPU is slow.** An all-reduce waits for its slowest participant. One overheating GPU slows the whole run, and finding it needs per-GPU timing data.
- **The data pipeline cannot keep up.** At high utilisation, GPUs consume data quickly. Tokenise and shuffle the data in advance, rather than during training.

How big should the model be?

Given a compute budget $C \approx 6ND$, how should you divide it between parameters N and tokens D? Hoffmann et al. (2022), in the paper known as Chinchilla, found that both should grow at about the same rate. The compute-optimal ratio is roughly 20 training tokens per parameter.

Combining $D = 20N$ with $C = 6ND$ gives $C = 120N^2$, so $N = \sqrt{C/120}$. For GPT-3's compute budget, that suggests 51 billion parameters trained on 1,025 billion tokens. GPT-3 instead used 175 billion parameters and 300 billion tokens.

The paper's own evidence was a direct comparison. Chinchilla, with 70 billion parameters trained on 1.4 trillion tokens, used about the same compute as Gopher, with 280 billion parameters. Chinchilla had four times fewer parameters, and it outperformed Gopher.

One caveat is worth raising. Chinchilla's rule minimises *training* compute. A model served to millions of users spends far more compute on inference over its lifetime. So it can be sensible to train a smaller model

for much longer than the Chinchilla ratio suggests, as the Llama models do.

Recap

- Mixed-precision training with Adam stores 16 bytes per parameter. 32-bit master weights exist because 16-bit numbers cannot absorb small updates.
- Activation memory grows with the square of sequence length, because of the attention scores. Recomputation trades time for memory.
- Training compute is about $6ND$ FLOPs.
- GPUs exchange data through collectives, and an all-reduce is a reduce-scatter plus an all-gather. The ring algorithm sends $2(n-1)/n$ copies per GPU, which stops growing at two.
- Data parallelism copies the model; ZeRO splits its stored state. Stages 1 and 2 cost no extra traffic; stage 3 costs half as much again. Tensor parallelism splits matrices, and needs fast links. Pipeline parallelism splits layers, and costs a bubble.
- Combine them so the heaviest communication uses the fastest links.
- The compute-optimal model uses about 20 tokens per parameter, but inference costs can justify a smaller model.

Reference models and conventions

Three set-ups appear in the article. Each is used where its numbers are public or round.

Where	The set-up	Why this one
sections 1 and 3	100 billion parameters on 64 GPUs	round numbers for the memory arithmetic
sections 2 and 4	GPT-3: 175 billion parameters, 96 layers, hidden size 12,288, 96 heads	its shape and token count are published
the exercise	70 billion parameters on 512 H100s	the configuration you will plan yourself

The letters follow the papers these formulas come from, so some of them differ from earlier modules. Section 9 of `GLOSSARY.md` maps the whole course.

Letter	Means	Elsewhere
N	number of parameters	
D	number of training tokens	
h	hidden size, the model width	d_{model}, or C in module 02
L	number of layers	
s	sequence length	T in modules 00 to 02
b	micro-batch size	
a	number of attention heads	H in module 02
n	number of GPUs in a group	
p, m	pipeline stages, micro-batches	

The exercise: parallelism, verified on one machine

What you will build. You will implement each parallelism scheme in a single process, and check numerically that it gives the same gradients as the unsplit model. A split computation either matches or it does not, and a laptop can answer that question. The exercise has five parts:

1. calculators for training memory, ZeRO, activation memory, compute and the Chinchilla size;
2. tensor-parallel MLP and attention layers;
3. data parallelism and gradient accumulation;
4. a pipeline scheduler that reproduces the bubble formula, rather than assuming it;
5. activation checkpointing, with its memory and time measured.

Time and setup. Allow three to four hours. The setup is the same as for module 00.

How to run it. Open `04-ml-system-design/exercise.ipynb`, or run `python exercise.py` from inside the module folder.

How the checks work. They work as in module 00. Before you start, each part reports that it is not implemented:

```
[FAIL] 1a  -- NotImplementedError:

0/1 passed.  Still failing: 1a
```

Step map. This table links each part to the section of the article it practises.

Part	You write	The idea it practises	Section
1a–1b	`training_memory`, `zero_memory`	the static memory budget, ZeRO	1, 3
1c	`activation_memory`	activations and recomputation	2
1d	`training_flops`, `chinchilla_optimal`	compute and model size	2, 7
2	`all_reduce`, `column_parallel_linear`, `row_parallel_linear`, `parallel_mlp`, `parallel_attention`	tensor parallelism	4
3	`data_parallel_grad`, `accumulated_grad`, `ring_allreduce_bytes`	data parallelism	3
4	`simulate_pipeline`, `gpipe_bubble_formula`	pipeline schedules	5
5	the checkpointed branch of `DeepMLP.forward`	activation recomputation	2

Several parts end with a written question. The most useful is 1e: plan a configuration for a 70-billion-parameter model on 512 H100s, and justify every number.

When you are done. The memory calculators reproduce the budgets in sections 1 to 3. Your split MLP and attention layers produce the same forward results and gradients as the unsplit versions, to within 10^{-10}. Your pipeline simulator reproduces the bubble formula exactly for every combination tested, and shows 1F1B storing far fewer activations. Checkpointing leaves the gradients unchanged.

If you get stuck.

- For tensor parallelism, work with two GPUs and write the shape of every shard on paper first.
- If a sharded result is wrong by a constant factor, check whether you summed where you should have averaged, or the reverse.

- For the pipeline simulator, print the schedule grid, as in section 5, and check each dependency by eye.
- If you are still stuck, `solution.py` has a complete reference.

Want more? The notebook ends with four stretch goals: simulate interleaved 1F1B, add communication time to the simulator, simulate ZeRO-3, and turn the memory calculator into a tool that proposes a configuration.

Going deeper

Start here

- Narayanan et al. (2021), *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*, SC '21. arXiv:2104.04473. How tensor, pipeline and data parallelism combine in practice, with measurements at scale. Read it after sections 4 to 6.

Parallelism

- Shoeybi et al. (2019), *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv:1909.08053. Section 3 gives the column-then-row split from section 4 in full.
- Rajbhandari et al. (2020), *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*, SC '20. arXiv:1910.02054. Section 5 gives the stage-by-stage memory accounting.
- Zhao et al. (2023), *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. arXiv:2304.11277. How ZeRO-3 is implemented in PyTorch.
- Huang et al. (2019), *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*, NeurIPS. arXiv:1811.06965. The original pipeline schedule and its bubble.
- Harlap et al. (2018), *PipeDream: Fast and Efficient Pipeline Parallel DNN Training*. arXiv:1806.03377. The 1F1B schedule.
- Liu et al. (2023), *Ring Attention with Blockwise Transformers for Near-Infinite Context*. arXiv:2310.01889.
- Fedus, Zoph & Shazeer (2022), *Switch Transformers*, JMLR 23. arXiv:2101.03961. A clear introduction to mixture-of-experts models.

Memory

- Korthikanti et al. (2022), *Reducing Activation Recomputation in Large Transformer Models*. arXiv:2205.05198. The activation-memory formula from section 2, and selective recomputation.
- Chen et al. (2016), *Training Deep Nets with Sublinear Memory Cost*. arXiv:1604.06174. The original gradient-checkpointing paper.

Scaling and model size

- Kaplan et al. (2020), *Scaling Laws for Neural Language Models*. arXiv:2001.08361.
- Hoffmann et al. (2022), *Training Compute-Optimal Large Language Models*, NeurIPS. arXiv:2203.15556. The Chinchilla paper from section 7.
- Sardana & Frankle (2023), *Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws*. arXiv:2401.00448. The inference-aware correction to Chinchilla.

What really happens at scale

- Zhang et al. (2022), *OPT: Open Pre-trained Transformer Language Models*. arXiv:2205.01068. Its published training logbook is the most candid public account of what goes wrong.

- Chowdhery et al. (2022), *PaLM: Scaling Language Modeling with Pathways*. arXiv:2204.02311. Section 4 covers hardware failures and loss spikes during a real run.
- Dubey et al. (2024), *The Llama 3 Herd of Models*. arXiv:2407.21783. Section 3.3 reports infrastructure and failure rates on 16,000 GPUs.
- NVIDIA, *NCCL documentation* — https://docs.nvidia.com/deeplearning/nccl/. The collective-communication library behind all-reduce, and its algorithms.

Next: module 05 takes a trained model and asks how to make it answer quickly.